Đại học Quốc gia Thành phố Hồ Chí Minh

Đại học Khoa học tự nhiên

Khoa Công nghệ thông tin

HOMEWORK

DATA GENERATION & SCENARIO TESTING

Subject **SOFTWARE TESTING**

Class 22KTPM3

Student **22127374 – Lê Thanh Tâm**

Ho Chi Minh City, 2025

Table of Contents

1 Group Task Allocation

Fullname	StudentID	Table 1	Table 2
Lương Hoàng Dung	22127076	products	contact_requests
Lê Thanh Tâm	22127374	invoice_items	categories
Nguyễn Lâm Nhã Uyên	22127445	users	favorites
Lê Nguyễn Yến Vy	22127466	invoices	contact_request_replies

2 Introduction

This report details the process of generating test data for "The Toolshop" application. The objective is to create a large volume of realistic and meaningful data to support comprehensive software testing activities, as required by the homework assignment. The data generation process is tailored to facilitate in-depth testing of specific features handled in previous assignments.

3 Table and Tool Selection

3.1 Selected Tables and Rationale

To test the application's core functionalities, two critical database tables were selected for data generation: `categories` and `invoice_items`.
The selection of these tables is a direct continuation of the work performed in **Homework #02: Domain Testing**, where my assigned responsibilities included testing features related to **orders, products, and categories**. Therefore, generating rich and varied data for these specific tables is essential for thoroughly testing the interconnected workflows, such as managing category structures, adding products to an invoice, and calculating order totals.

- `categories`: This table is fundamental to product organization and directly impacts user navigation, filtering, and search functionalities. Populating it with a hierarchical structure (parent-child relationships) allows for testing edge cases in category management.
- `invoice_items`: As a central pivot table, it links products to invoices and is critical for the entire sales process. Generating extensive data for this table enables testing of calculations, data integrity, and performance under load.

3.2 Data Generation Tool

To meet the requirement of generating meaningful and highly customizable data, a **custom-built tool** was developed. This tool consists of a **Python script** that leverages powerful libraries:

- **Faker**: To generate a wide variety of realistic fake data (names, dates, numbers, etc.).
- **Pandas**: To efficiently process and export the generated data into an Excel file.

This approach was chosen because it offers maximum flexibility to define complex generation rules, ensure data uniqueness (e.g., for `slugs`), and create data that is thematically consistent with "The Toolshop" application's context.

4 Data Fields and Generation Rules

The following tables outline the data fields and their corresponding randomization rules for each selected table.

4.1 Table 1: `categories` (500 rows)

Field Name	Data Type	Generation Rule / Range	Notes
`id`	Integer	Auto-incremented from 1 to 500.	Primary Key
`parent_id`	Integer (Nullable)	30% chance of being assigned an existing category `id`. `NULL` for the remaining 70%.	Creates a parent-child hierarchy
`name`	String	A realistic, 2-4 word category name, themed for a tool shop (e.g., "Power Tools Accessories").	Name is unique
`slug`	String	Automatically generated from the `name` field, lowercased, with spaces replaced by hyphens.	Slug is unique
`created_at`	Timestamp	A random timestamp within the last 3 years.	
`updated_at`	Timestamp	Same value as `created_at`.	

4.2 Table 2: `invoice_items` (500 rows)

Field Name	Data Type	Generation Rule / Range	Notes
`id`	Integer	Auto-incremented from 1 to 500.	Primary Key
`invoice_id`	Integer	A random integer between 1 and 100.	Assumes 100 invoices exist
`product_id`	Integer	A random integer between 1 and 50.	Assumes 50 products exist
`unit_price`	Decimal (2 digits)	A random decimal number between 10.00 and 500.00.	Price per unit
`quantity`	Integer	A random integer between 1 and 10.	Item quantity
`created_at`	Timestamp	A random timestamp within the last 2 years.	
`updated_at`	Timestamp	Same value as `created_at`.	

4.3 Rationale for Timestamp Columns (`created_at`, `updated_at`)

A careful observation of the application's UI reveals that date and time information for categories and invoice items is not displayed to the end-user. However, the `created_at` and `updated_at` columns are intentionally included in the data generation process. This is a standard and critical practice in modern software development for several reasons:

- **Auditing and Logging:** These timestamps provide an essential audit trail for administrators and developers to track when a record was created and last modified.
- **Backend Business Logic:** The backend can leverage these timestamps for features not visible on the UI, such as sorting items by "Newest" or generating time-based reports.
- **Data Integrity and Synchronization:** Timestamps are crucial for resolving data conflicts and managing data synchronization in complex systems.
- **Framework Conventions:** Many popular web frameworks (like Laravel) automatically manage these columns. Including them ensures our test data aligns with the real application schema.

In summary, their absence on the UI is a **design decision**, while their presence in the database is a **technical requirement**.

5 Process and Steps

1. **Requirement Analysis:** The assignment file was analyzed to understand the table structures and data constraints.
2. **Script Development:** A Python script was created to automate data generation.
3. **Script Execution:** The script was executed from the terminal using `python` **22127374_DataGeneration_Data.py**.
4. **Data Export:** The script saved the data into an Excel file with two sheets, which was then renamed for submission.

6 Source Code Explanation

6.1 Full Script

The complete source code for the custom-built, optimized data generation tool is provided below.

```
import pandas as pd
from faker import Faker
import random
from datetime import datetime, timedelta

# --- Configuration ---
NUM_CATEGORIES = 500
NUM_INVOICE_ITEMS = 500
OUTPUT_FILENAME = 'generated_data.xlsx'
```

```
10
11 # --- Initialize Faker ---
12 fake = Faker()
13
14 # --- Helper Functions ---
15 def generate_slug(name):
16     """Generates a URL-safe slug from a string."""
17     # Updated to handle model codes in parentheses
18     return name.lower().replace(' & ', ' ').replace('(', '').replace(')', '').
   replace(' ', '-')
19
20 def generate_model_code():
21     """Generates a realistic-looking model code."""
22     prefix = random.choice(['Pro', 'HD', 'XT', 'MK-II', 'Series', 'G'])
23     number = random.randint(100, 9500)
24     return f"({prefix}-{number})"
25
26 def generate_tool_category_name():
27     """Generates a realistic, tool-shop-themed category name with a model code.
   """
28     tool_types = ['Power Tools', 'Hand Tools', 'Gardening Tools',
29                   'Automotive Tools', 'Woodworking', 'Metalworking', 'Plumbing']
30     specific_tools = ['Drills', 'Saws', 'Wrenches', 'Hammers', 'Pliers',
31                       'Screwdrivers', 'Sockets', 'Clamps', 'Sanders', 'Grinders']
32     attributes = ['Heavy Duty', 'Precision', 'Portable', 'Cordless',
33                   'Professional', 'DIY']
34     suffixes = ['Kits', 'Accessories', 'Supplies', 'Storage', 'Equipment', '
   Safety Gear']
35
36     methods = [
37         lambda: f"{random.choice(attributes)} {random.choice(specific_tools)}",
38         lambda: f"{random.choice(tool_types)} {random.choice(suffixes)}",
39         lambda: f"{random.choice(specific_tools)} & {suffixes[random.randint(0,
   2)]}",
40         lambda: f"Essential {random.choice(tool_types)}",
41         lambda: f"{random.choice(specific_tools)} Sets"
42     ]
43
44     base_name = random.choice(methods)()
45     # Append a realistic model code to ensure uniqueness and realism
46     model_code = generate_model_code()
47
48     return f"{base_name} {model_code}"
49
50 # --- 1. Generate 'categories' data ---
51 print(f"Generating {NUM_CATEGORIES} categories...")
52 categories_data = []
53 generated_slugs = set()
54
55 for i in range(1, NUM_CATEGORIES + 1):
56     # The while loop is now just a safety net for extremely rare collisions.
57     # It should run very fast.
58     while True:
59         name = generate_tool_category_name()
60         slug = generate_slug(name)
61         if slug not in generated_slugs:
62             generated_slugs.add(slug)
```

```
            break

    # Assign parent_id: 30% chance of having a parent
    parent_id = None
    if i > 1 and random.random() < 0.3:
        parent_id = random.randint(1, i - 1)

    created_at = fake.date_time_between(start_date='-3y', end_date='now')
    updated_at = created_at

    categories_data.append({
        'id': i,
        'parent_id': parent_id,
        'name': name,
        'slug': slug,
        'created_at': created_at,
        'updated_at': updated_at
    })

print("Categories generation complete.")


# --- 2. Generate 'invoice_items' data ---
print(f"\nGenerating {NUM_INVOICE_ITEMS} invoice items...")
invoice_items_data = []

for i in range(1, NUM_INVOICE_ITEMS + 1):
    created_at = fake.date_time_between(start_date='-2y', end_date='now')
    updated_at = created_at

    invoice_items_data.append({
        'id': i,
        'invoice_id': random.randint(1, 100),
        'product_id': random.randint(1, 50),
        'unit_price': round(random.uniform(10.00, 500.00), 2),
        'quantity': random.randint(1, 10),
        'created_at': created_at,
        'updated_at': updated_at
    })

print("Invoice items generation complete.")


# --- 3. Export to Excel ---
print(f"\nExporting data to {OUTPUT_FILENAME}...")
try:
    with pd.ExcelWriter(OUTPUT_FILENAME) as writer:
        categories_df = pd.DataFrame(categories_data)
        invoice_items_df = pd.DataFrame(invoice_items_data)

        categories_df.to_excel(writer, sheet_name='categories', index=False)
        invoice_items_df.to_excel(writer, sheet_name='invoice_items', index=False
    )

    print(f"\nSuccessfully created {OUTPUT_FILENAME} with two sheets: 'categories
    ' and 'invoice_items'.")

```

```
except Exception as e:
    print(f"\nAn error occurred while writing to the Excel file: {e}")
```

Listing 1: Python Script for Data Generation

6.2 Performance Optimization: Ensuring Uniqueness

An important update was made to the script to resolve a performance bottleneck. The initial version of the script used a `while True:` loop to ensure that every generated category `name` and `slug` was unique. While logically correct, this approach became extremely slow as the number of generated categories increased, because the probability of randomly generating a duplicate name grew higher. This caused the script to "hang" as it repeatedly failed to find a unique name.
To solve this, the `generate_tool_category_name` function was modified to append a **unique, realistic-looking model code** (e.g., `(Pro-550)`, `(MK-II)`) to each base category name. This strategy offers two key advantages:

1. **Performance:** It drastically reduces the chance of a name collision, making the `while` loop a simple safety check that rarely needs to iterate more than once. This allows the script to execute in seconds, not minutes.

2. **Realism:** The resulting data (e.g., `Heavy Duty Drills (Pro-550)`) looks more authentic and professional, mimicking real-world product naming conventions and improving the overall quality of the test data.

7 Sample Data

7.1 `categories` Table

```
id,parent_id,name,slug,created_at,updated_at
1,NULL,"Precision Sockets (Pro-1234)","precision-sockets-pro-1234",...
2,NULL,"Essential Metalworking (HD-5678)","essential-metalworking-hd-5678",...
3,1,"Automotive Equipment (XT-2233)","automotive-equipment-xt-2233",...
... (497 more rows)
```

7.2 `invoice_items` Table

```
id,invoice_id,product_id,unit_price,quantity,created_at,updated_at
1,87,23,154.55,3,2024-03-15...,2024-03-15...
2,54,12,489.99,1,2023-09-02...,2023-09-02...
3,21,45,78.50,5,2025-01-10...,2025-01-10...
... (497 more rows)
```

8 Use of AI Tools

An AI tool was used extensively to assist in the completion of this assignment. This section provides a detailed, step-by-step account of the interaction.

8.1 AI Tool Name

- **Gemini** (Google's Large Language Model)
- **Grok**
- **ChatGPT**

8.2 Detailed Interaction with AI

The process of generating the final script was iterative, involving multiple prompts to refine the output.

8.2.1 Step 1: Providing Context

The first step was to provide the AI with the necessary context, which was the database schema for the two tables. This ensures the AI understands the data structure it needs to work with.

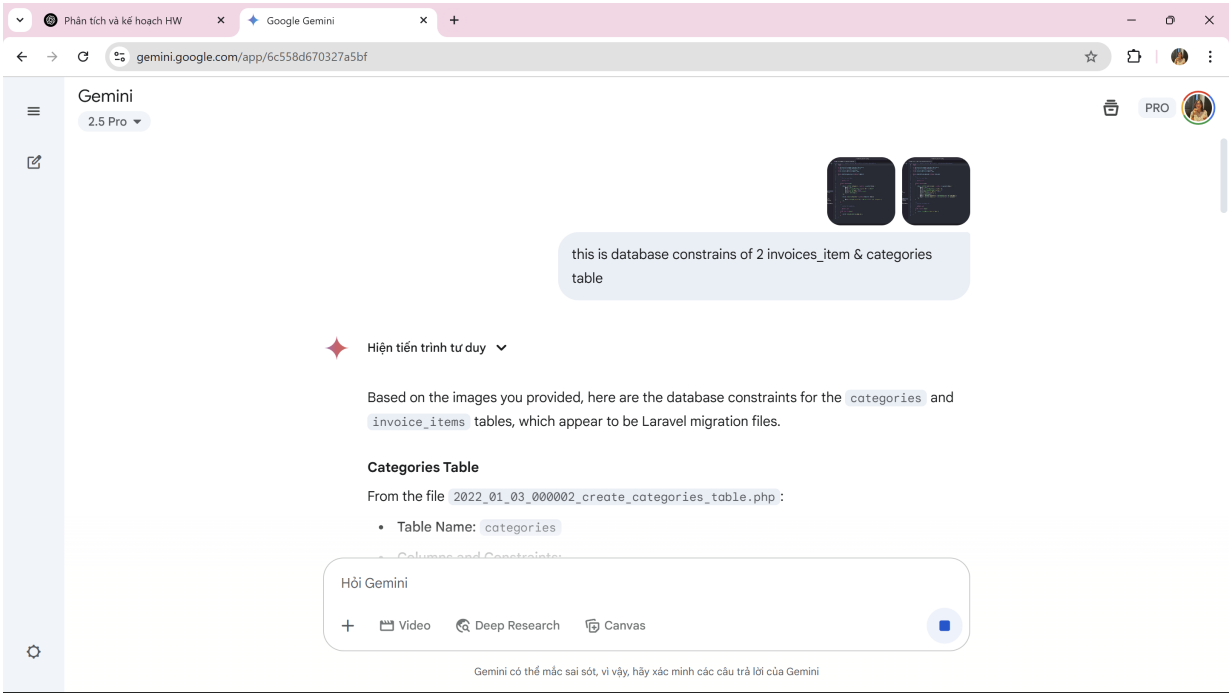

Figure 1: Providing database constraints as context for the AI.

8.2.2 Step 2: Initial Data Generation Request

To quickly assess the AI's ability to generate meaningful data, a direct prompt was used to request 500 rows for the `invoice_items` table in CSV format.

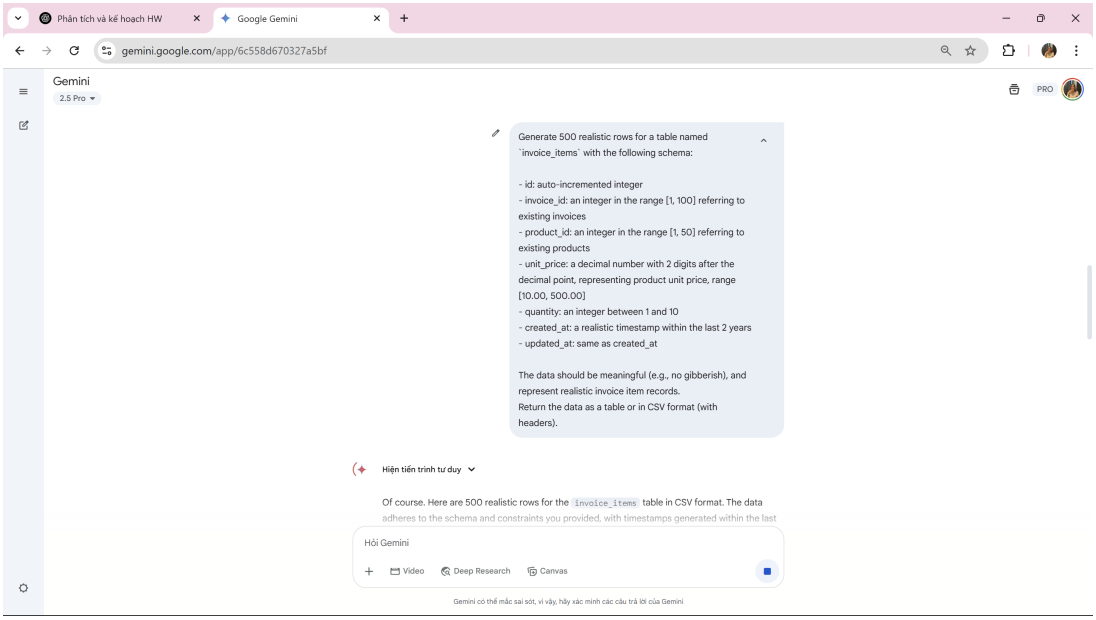

Figure 2: Prompt requesting the generation of 500 rows for the `invoice_items` table.

The AI successfully generated the data in the requested format, demonstrating its understanding of the schema. The same steps with **categories** table.

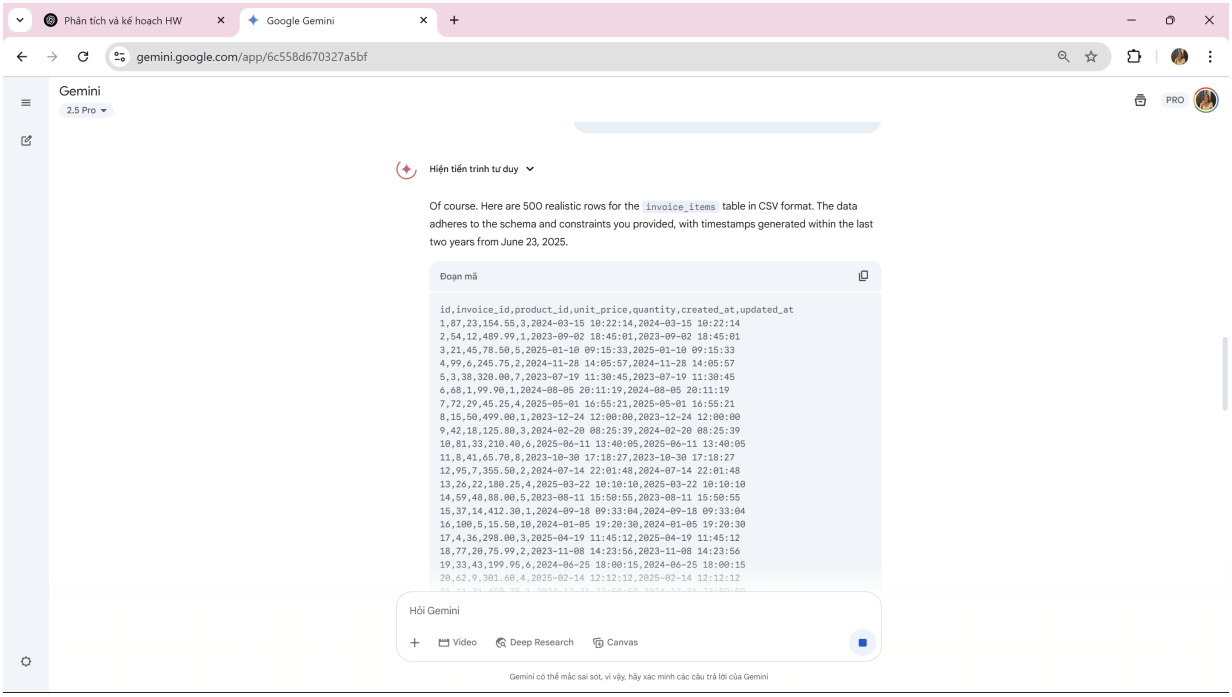

Figure 3: Sample of the raw CSV data generated by the AI.

8.2.3 Step 3: Requesting Automation via Scripting

Realizing that generating data directly and then manually copying it was inefficient and inflexible, the next step was to request a reusable tool. The prompt was updated to ask for a complete Python

script using the Faker library to automate the entire process for both tables and export the result to an Excel file.

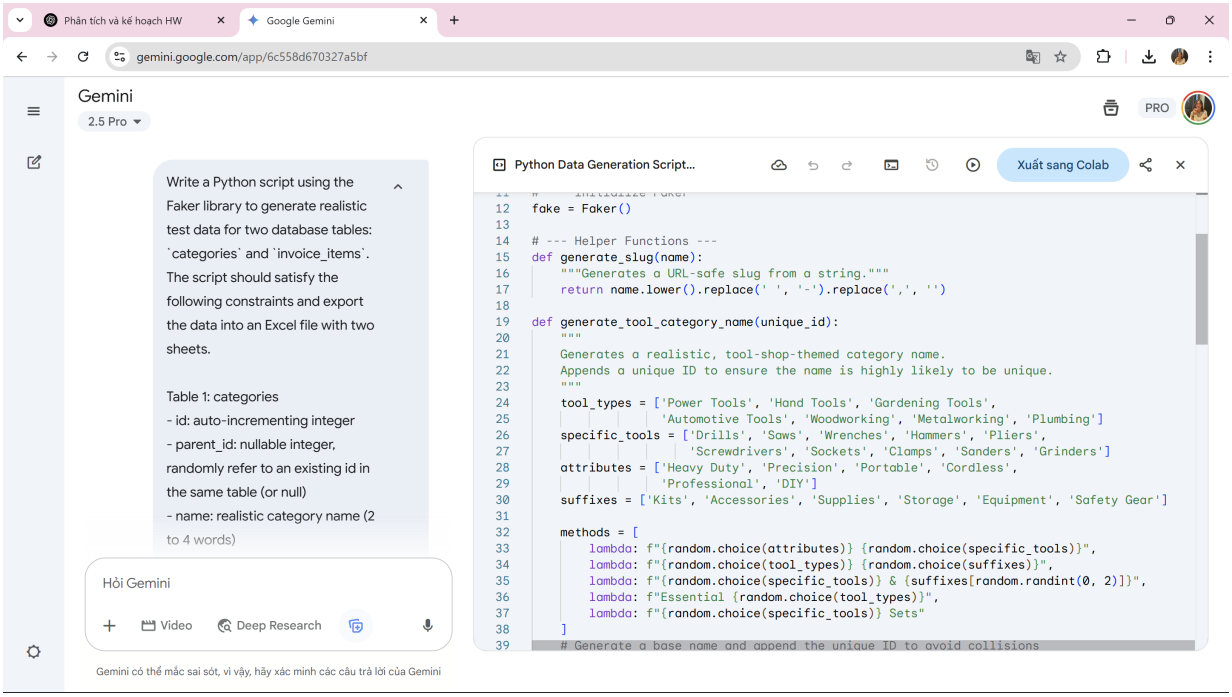

Figure 4: Prompt requesting a full Python script for automation.

8.2.4 Step 4: Refinement and Optimization

The script generated by the AI was functional but had two issues: the data was too generic, and it ran very slowly. This led to further refinement prompts:

- **Contextual Refinement:** Screenshots of "The Toolshop" UI were provided with the prompt: *"based on this example, update the script to generate data more suitable for a tool shop theme"*.
- **Performance Optimization:** After observing the slow execution, a final prompt was used: *"The script is running very slow. How can I optimize the unique name generation?"*. This led to the final, efficient version of the script detailed in Section 5.

8.3 Validation and Refinement Process

The interaction with the AI was an iterative dialogue. It began with broad requests, which were progressively narrowed down by providing more specific context (the application's theme, UI, and performance issues). The final script was manually reviewed for logical correctness, and the output data was verified to ensure it was meaningful, realistic, and adhered to all constraints.

9 Self-Assessment

Criteria	Description	Max Points	Self-Assessed Points
2 Tables Selection	2 important tables selected	1.0	1.0
Sample data	All data must be **meaningful**	2.0	2.0
Data generation report	Report is clear, traceable, professional, with self-assessment	1.0	1.0
Total (Data Generation)		**4.0**	**4.0**